

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего
образования

«Владивостокский государственный университет»

Колледж информационных и креативных технологий

ДИПЛОМНЫЙ ПРОЕКТ

**Тема: «Разработка веб-приложения SysAssist для автоматизации
обработки ИТ-инцидентов и управления регламентированными
действиями в инфраструктуре предприятия»**

Специальность: 09.02.07 Информационные системы и программирование

Квалификация: программист

Студент	_____ Д.В. Пыхтин
Руководитель	_____ А.А. Тюбаев
Нормоконтролер	_____ _____

Владивосток 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 АНАЛИТИЧЕСКОЕ ОБОСНОВАНИЕ И ПОСТАНОВКА ЗАДАЧИ	5
1.1 Предметная область IT-operations и проблема регламентированных действий	5
1.2 Анализ подходов, требования, стек и риски	6
2 ПРОЕКТНО-ТЕХНИЧЕСКАЯ РЕАЛИЗАЦИЯ SYSASSIST	12
2.1 Архитектура решения и состав репозитория	12
2.2 API, данные, модули, безопасность и frontend	14
3 ТЕСТИРОВАНИЕ, ЭКСПЛУАТАЦИЯ И ОЦЕНКА РЕЗУЛЬТАТА	25
3.1 Методика тестирования	25
3.2 Диагностика, развертывание и практическая оценка	26
ЗАКЛЮЧЕНИЕ	31
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	32
ПРИЛОЖЕНИЯ	33

ВВЕДЕНИЕ

Современная инфраструктура предприятия состоит из множества наблюдаемых компонентов: серверов, баз данных, HTTP-сервисов, прокси, систем мониторинга, каналов уведомлений и внутренних регламентов.

Практическая сложность заключается не только в получении сигналов от Zabbix, Grafana, Prometheus Alertmanager или локальных проверок, но и в согласованном переводе этих сигналов в инциденты, рекомендации, действия, аудит и доказательную базу сопровождения.

Актуальность проекта SysAssist определяется необходимостью создать on-premise веб-приложение, которое не подменяет существующие средства мониторинга, а связывает их в управляемый операционный контур. В таком контуре событие должно быть принято, нормализовано, классифицировано, связано с модулем-источником, дополнено рекомендацией, передано на согласование при высоком риске и зафиксировано в журнале. Такой подход уменьшает долю ручной координации и повышает воспроизводимость работ сопровождения.

Цель дипломного проекта - разработать и описать веб-приложение SysAssist для автоматизации обработки ИТ-инцидентов и управления регламентированными действиями в инфраструктуре предприятия. Под продуктовым состоянием в работе понимается не демонстрационный макет, а система с реальной базой данных, защищенными секретами, лицензированием, диагностикой, поддержкой модулей, smoke-проверками и интерфейсом оператора.

Объектом исследования является процесс сопровождения программного обеспечения и инфраструктурных сервисов предприятия. Предметом исследования является программная система, которая обеспечивает прием событий, модульную интеграцию, контролируемое выполнение действий, аудит и эксплуатационную диагностику.

Для достижения цели были решены задачи: проанализировать предметную область IT-operations; сформулировать требования к платформе; выбрать технологический стек; реализовать серверный API, модель данных и модульный реестр; разработать пользовательский интерфейс; внедрить защиту секретов и лицензирование; выполнить тестирование; подготовить production-readiness критерии и support bundle.

Методическую основу работы составляют принципы Clean Architecture, REST API, ролевой модели доступа, безопасного хранения секретов, событийной обработки инцидентов и регламентированной эксплуатации. Практическая часть основана на локальном репозитории SysAssist, в котором реализованы backend на .NET, frontend на React/Vite, база данных CockroachDB через EF Core/Npgsql и набор интеграционных адаптеров.

Практическая значимость работы состоит в том, что результат может использоваться как основа для внутренней платформы сопровождения: оператор видит состояние модулей и событий, инженер управляет диагностикой и настройками, администратор контролирует пользователей и модули, аудитор выгружает support bundle, а старший администратор принимает решения по опасным действиям.

Ключевая идея проекта

SysAssist проектируется как координационный слой между мониторингом, инфраструктурой, регламентами и аудитом. Ценность системы не в отдельном dashboard, а в трассируемой цепочке: сигнал -> инцидент -> рекомендация -> согласование -> действие -> журнал -> доказательная выгрузка.

Таблица 1 - Основные характеристики дипломного проекта

Параметр	Содержание
Тип системы	On-premise веб-приложение для автоматизации IT-operations
Backend	.NET 9, ASP.NET Core Minimal API, Serilog, JWT Bearer
Frontend	React, TypeScript, Vite, TanStack Query, Zustand, Recharts
База данных	CockroachDB через PostgreSQL-compatible wire protocol и EF Core/Npgsql

Безопасность	RBAC, JWT, AES-GCM для секретов, audit trail, rate limiting
Интеграции	13 модулей: мониторинг, БД, инфраструктура, уведомления, advisor
Проверка готовности	health/ready, diagnostics, production-readiness gates, prod-smoke

1 АНАЛИТИЧЕСКОЕ ОБОСНОВАНИЕ И ПОСТАНОВКА ЗАДАЧИ

1.1 Предметная область IT-operations и проблема регламентированных действий

IT-operations представляет собой совокупность процессов, обеспечивающих стабильную работу информационных систем: мониторинг, обнаружение отказов, первичная диагностика, эскалация, выполнение регламентированных действий и документирование результата. В реальной инфраструктуре эти процессы часто распределены между несколькими инструментами и несколькими ролями, что создает задержки и повышает вероятность ошибки.

Типичная проблема состоит в разрыве между сигналом мониторинга и управляемым действием. Система мониторинга может зарегистрировать недоступность HTTP endpoint, рост задержек, ошибку в Nginx, сбой базы данных или превышение порога ресурсов. Однако сама по себе запись события не отвечает на вопросы: кто ответственен, насколько действие опасно, требуется ли согласование, какие данные войдут в аудит и как подтвердить корректность реакции.

Регламентированное действие отличается от обычной реакции тем, что оно должно быть воспроизводимым и контролируемым. Перезапуск контейнера, изменение конфигурации, отправка уведомления, выполнение диагностики или сбор support bundle требуют ясного контекста. Для высокорисковых операций необходимо согласование, а для безопасной демонстрации - режим SafeMode, при котором действие симулируется и честно записывается как симуляция.

SysAssist закрывает данный разрыв за счет единого жизненного цикла события. Событие попадает в платформу через webhook или модульный адаптер, нормализуется в инцидент, получает рекомендацию от локального rule advisor, связывается с действием и проходит через workflow согласования. Это переводит реакцию из неформальной переписки в документируемый процесс.

Для дипломного проекта особенно важно, что система не использует фиктивную диагностику как финальный результат. В продуктовой версии диагностика должна опираться на реальные adapter health checks, указывать свежесть проверки и показывать production gates. Такой подход позволяет отличить красивый экран от эксплуатационно пригодной системы.

1.2 Анализ существующих подходов к мониторингу и обработке инцидентов

На практике предприятия используют специализированные средства мониторинга: Zabbix для инфраструктурных триггеров, Grafana для визуализации и alerting, Prometheus Alertmanager для маршрутизации alert-событий, Nginx status для прокси-метрик, PostgreSQL/Redis checks для сервисов данных. Эти инструменты эффективны в своей предметной области, но не всегда образуют единую цепочку выполнения регламентов.

Второй распространенный подход - ручная обработка событий через мессенджеры, почту и таблицы. Он прост в запуске, но плохо масштабируется: информация теряется, согласования не имеют строгих статусов, а аудит формируется постфактум. При росте количества модулей и событий такой подход приводит к операционному шуму и снижению управляемости.

Третий подход - внедрение крупных ITSM/ITOM-платформ. Он дает формальные процессы, но требует сложной настройки, интеграции с внешними сервисами и значительных затрат. Для учебного и малого корпоративного контура целесообразно разработать более компактную

систему, сохранив ключевые свойства: ролевой доступ, нормализацию событий, согласование действий, журналы, диагностику и выгрузку доказательств.

SysAssist занимает промежуточную позицию. Система не конкурирует с мониторингом и не пытается заменить все ITSM-функции. Она выполняет роль координационного слоя: собирает события из разных источников, приводит их к единой модели, добавляет локальные рекомендации, контролирует риск действия и предоставляет интерфейс для оператора, инженера, администратора и аудитора.

Сравнение подходов показывает, что для дипломного проекта критичны модульность, возможность локального запуска, честное отображение состояния интеграций, безопасность секретов и возможность доказать результат тестами. Именно поэтому в SysAssist используются CockroachDB, EF Core, JWT/RBAC, AES-GCM для секретов, production-readiness endpoint и smoke-скрипт.

Таблица 2 - Сравнение подходов к обработке ИТ-инцидентов

Подход	Преимущества	Ограничения	Применимость в SysAssist
Только мониторинг	Быстрое обнаружение событий	Нет workflow действий и согласований	Используется как источник сигналов
Ручная координация	Не требует разработки	Нет надежного аудита и контроля риска	Заменяется формальным workflow
Крупная ITSM-платформа	Зрелые процессы	Высокая стоимость и сложность внедрения	Частично заимствованы принципы
Модульный координатор	Баланс контроля, локальности и расширяемости	Требует разработки адаптеров	Выбранный подход проекта

1.3 Требования к продукту SysAssist

Функциональные требования к SysAssist включают прием событий, просмотр dashboard, управление модулями, запуск health checks, получение событий, выполнение действий, обработку согласований, просмотр аудита,

журналов и уведомлений, а также экспорт support bundle. Система должна предоставлять единый интерфейс, в котором пользователь видит состояние инфраструктурных источников и результат операций.

Требование к модульности заключается в том, что каждый источник описывается через реестр модулей. Модуль имеет ключ, название, тип, описание, признаки polling/webhook/actions, состояние включения, health status, SafeMode, fallback mode и набор настроек. Такой подход упрощает расширение: добавление нового источника не должно ломать общий workflow.

Нефункциональные требования включают надежность, трассируемость, безопасность и пригодность к эксплуатации. Для надежности используется readiness endpoint и проверка подключения к CockroachDB. Для трассируемости применяются correlation id, audit entries, system logs и support bundle. Для безопасности внедрены JWT, RBAC, маскирование секретов, замена секретов отдельным endpoint, rate limiting и ограничение размера тела запроса.

Особое требование - отсутствие фиктивных данных в продуктивном контуре. Fallback mode допустим как демонстрационный режим, но в реальном режиме включенные модули должны иметь параметры подключения и проходить adapter health checks. Production-readiness endpoint должен явно показывать, какие условия мешают выпуску системы в production.

Интерфейс должен быть плотным и операторским: dashboard, модули, события, действия, аудит, логи, support bundle и диагностика. Для такой системы не подходит маркетинговая посадочная страница; первый экран должен быть рабочей консолью с реальными счетчиками и статусами.

Таблица 3 - Ролевые сценарии SysAssist

Роль	Основные действия	Ограничения
Operator	Просмотр dashboard, событий и уведомлений	Нет доступа к настройкам

		модулей и пользователям
Engineer	Диагностика, health checks, fetch events, операции модулей	Не управляет пользователями и критическими согласованиями
Admin	Модули, пользователи, роли, аудит, support bundle	Действия высокого риска проходят через workflow
SeniorAdmin	Принятие решений по approval requests	Решение финально и фиксируется в аудите
Auditor	Просмотр audit/log/support bundle	Нет изменения операционного состояния

1.4 Обоснование технологического стека и базы данных

Backend выбран на C#/.NET 9 и ASP.NET Core Web API. Этот стек подходит для корпоративных систем за счет строгой типизации, развитой middleware-модели, встроенной поддержки dependency injection, JWT-аутентификации, OpenAPI-документации и тестирования. Minimal API позволяет компактно описать HTTP surface без избыточной MVC-структуры.

Frontend реализован на React, TypeScript и Vite. React обеспечивает компонентную модель интерфейса, TypeScript снижает риск ошибок в DTO и API-клиенте, Vite ускоряет сборку и локальную разработку. Для операторской консоли используются TanStack Query для серверного состояния, Zustand для состояния shell, Recharts для графиков, Framer Motion/GSAP для умеренной анимации и Lucide React для иконок.

В качестве базы данных используется CockroachDB, работающая через PostgreSQL-compatible wire protocol. Для приложения это означает возможность применять EF Core и Npgsql, сохраняя SQL-модель, транзакции, UUID-ключи, индексы и JSONB-поля. SQLite в проекте не используется, поскольку целевой контур требует production-oriented распределяемой SQL базы.

EF Core выбран как ORM, потому что он связывает доменные сущности с миграциями, типами перечислений, индексами и таблицами. В SysAssist модель данных включает инциденты, рекомендации, approval

requests, действия модулей, настройки, health history, webhook events, audit entries, system logs, notifications, пользователей, поли и support bundle exports.

Технологический стек также учитывает сопровождение. Serilog используется для структурированного логирования, Swagger/OpenAPI - для проверки API, xUnit - для автоматизированных тестов, PowerShell smoke script - для сквозной проверки после запуска. Такое сочетание делает проект не только демонстрационным, но и проверяемым.

Таблица 4 - Технологический стек проекта

Слой	Технологии	Назначение
API	.NET 9, ASP.NET Core, Minimal API	HTTP endpoints, middleware, auth, rate limiting
Application	C# interfaces, use-case contracts	Границы бизнес-сценариев и портов
Infrastructure	EF Core, Npgsql, CockroachDB, adapters	Данные, интеграции, auth, license, secrets
Frontend	React, TypeScript, Vite	Операторская консоль и forms/workflows
Security	JWT, RBAC, AES-GCM, ECDSA license	Доступ, секреты, лицензирование
QA	xUnit, prod-smoke.ps1, readiness gates	Автоматизированная проверка качества

1.5 Модель рисков, ролей и эксплуатационных ограничений

Система, выполняющая действия в инфраструктуре, должна исходить из принципа минимизации риска. Не каждое действие может быть выполнено сразу после рекомендации. Низкорисковые диагностические операции допустимы без approval, тогда как high/critical actions переводят событие в PendingApproval и требуют решения пользователя с ролью Admin или SeniorAdmin.

SafeMode является важным эксплуатационным ограничителем. Когда SafeMode включен, действие симулируется, но записывается честный результат ACTION_SIMULATED. Это позволяет демонстрировать workflow

и проверять интеграцию без риска перезапуска сервиса, изменения конфигурации или отправки нежелательного уведомления.

Секреты модулей относятся к критическим данным. API не возвращает значение секрета в открытом виде: вместо этого отображается маска и признак `hasValue`. Замена секрета выполняется отдельным endpoint, а хранение производится через `ISecretProtector`. В реальном режиме отсутствие `encryption key` является blocker-условием.

Лицензирование используется как контроль продуктового доступа. Лицензия проверяется по подписи, продукту, временному окну и `fingerprint`. При невалидной лицензии `protected API routes` блокируются, при этом `auth/license` и `production-readiness` остаются доступны для восстановления и диагностики.

`Production-readiness gates` фиксируют границу между локальной разработкой и `production`. Если приложение запущено в `Development`, использует `startup migrations`, wildcard `AllowedHosts` или локальный `CORS`, статус становится `Blocked`. Это не ошибка системы, а честное указание, что конфигурация не является продуктивной.

2 ПРОЕКТНО-ТЕХНИЧЕСКАЯ РЕАЛИЗАЦИЯ SYSASSIST

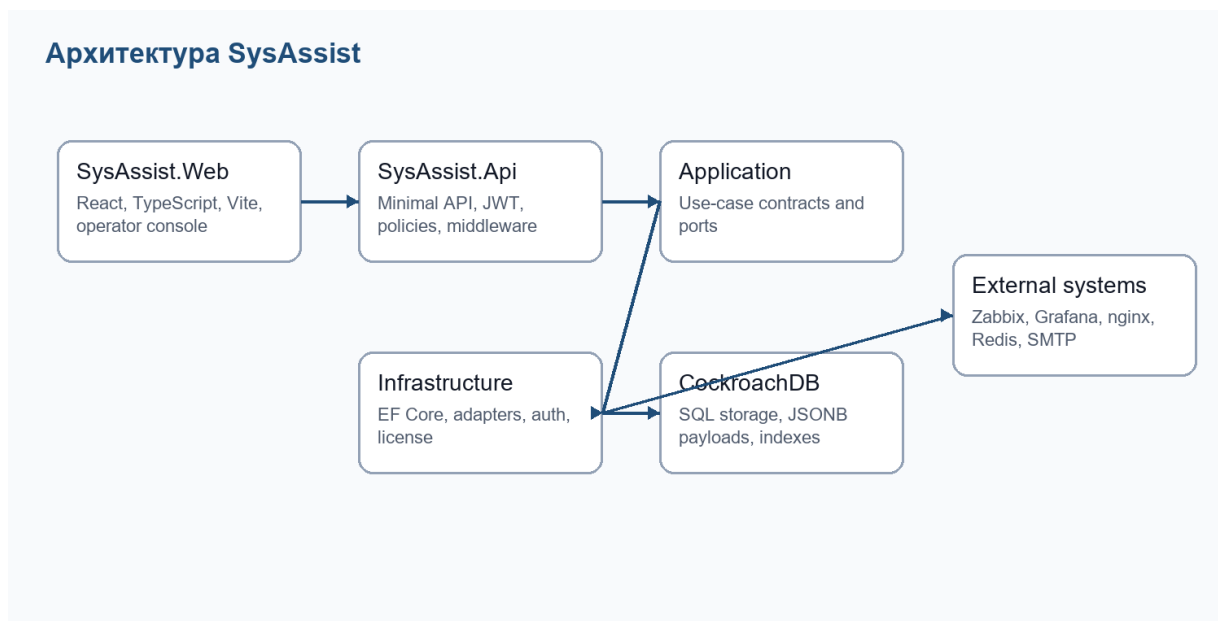


Рисунок 1 - Архитектура SysAssist

2.1 Архитектура решения и состав репозитория

Проект построен по Clean Architecture. Внутренний доменный слой не зависит от HTTP и базы данных, application layer задает контракты, infrastructure реализует хранение и интеграции, API слой собирает зависимости и публикует endpoints, frontend является отдельным клиентским приложением. Такая структура снижает связанность и упрощает тестирование.

Слой SysAssist.Domain содержит сущности, перечисления и базовую предметную модель. К ним относятся IncidentEvent, EventRecommendation, ApprovalRequest, IntegrationModule, IntegrationSetting, IntegrationModuleAction, AuditEntry, SystemLog, Notification, User, Role и SupportBundleExport. Эта модель задает язык приложения.

Слой SysAssist.Contracts содержит DTO, которые проходят через HTTP API и frontend client. Благодаря этому React-приложение работает с понятными структурами: DashboardDto, ModuleDto, DiagnosticsDto, ProductionReadinessDto, LicenseStatusDto и другими контрактами. Разделение DTO от EF-сущностей снижает риск случайной утечки внутренних полей.

Слой SysAssist.Infrastructure реализует EF Core DbContext, seed data, secret protection, license service, auth service, module catalog, adapter runtime и основной SysAssist.ApiService. Он является самым насыщенным технически, так как соединяет доменную модель с БД, внешними источниками и операционными правилами.

SysAssist.Api отвечает за composition root. Здесь конфигурируются Kestrel limits, Serilog, JWT Bearer, authorization policies, CORS, rate limiting, security headers, exception middleware, license wall, health endpoints и Minimal API routes. Эта точка важна для production hardening, потому что именно здесь проверяются критические настройки запуска.

SysAssist.Web реализует операторский интерфейс. В отличие от маркетинговых страниц, он сразу показывает рабочую консоль: dashboard, события, approvals, modules, actions, audit, logs, notifications, users/roles, support bundle и diagnostics. Такое построение соответствует назначению продукта.

Таблица 5 - Структура репозитория SysAssist

Каталог	Назначение
src/SysAssist.Domain	Предметные сущности, enums и доменная модель
src/SysAssist.Contracts	DTO для API и frontend-клиента
src/SysAssist.Application	Интерфейсы use-case сервисов, auth, security ports
src/SysAssist.Infrastructure	EF Core, CockroachDB, adapters, seeding, auth, license, secrets
src/SysAssist.Api	HTTP endpoints, middleware, policies, health/readiness
src/SysAssist.Web	React/Vite operator console
tests/SysAssist.Tests	xUnit тесты workflow, modules, auth, support bundle security
scripts/prod-smoke.ps1	Сквозная production smoke-проверка

2.2 Серверный API, контракты и middleware

HTTP API реализован как набор Minimal API групп. Такой подход позволяет явно видеть authorization policy каждого маршрута и не размазывать бизнес-операции по контроллерам. Основные группы включают

auth, dashboard, events, approvals, modules, custom-modules, actions, audit, logs, notifications, users, roles, diagnostics, support-bundle и webhooks.

Аутентификация строится на JWT Bearer. JwtOptions считываются из конфигурации, а token validation включает issuer, audience, lifetime и signing key. В production ключ подписи должен быть реальным секретом длиной не менее 32 символов; dev-only fallback запрещается в real mode без явного development allowance.

Authorization policies разделяют роли. RequireOperatorOrHigher открывает dashboard/events, RequireEngineerOrHigher - модули и диагностику, RequireAdmin - управление пользователями и модулями, RequireSeniorAdmin - критические approvals, RequireAuditorOrAdmin - audit/support bundle. Это снижает риск неконтролируемого доступа к опасным операциям.

Middleware добавляет correlation id, обработчик исключений, Serilog request logging, security headers, CORS, rate limiter, authentication, authorization и license wall. Correlation id помещается в response headers и audit/log записи, что помогает связать пользовательское действие, API-запрос и системный журнал.

Readiness endpoint проверяет CockroachDB и лицензию, а production-readiness endpoint формирует расширенный список gates. В отличие от обычного health endpoint, production-readiness не только сообщает, работает ли процесс, но и объясняет, может ли конфигурация считаться релизной.

Листинг 1 - Контракт production-readiness DTO

```
public sealed record ProductionReadinessDto(  
    string Status,  
    DateTimeOffset CheckedAt,  
    IReadOnlyCollection<ProductionReadinessGateDto> Gates);  
  
public sealed record ProductionReadinessGateDto(  
    string Key,  
    bool Passed,  
    string Message,  
    bool Required = true);
```

Листинг 2 - Регистрация защищенного endpoint production-readiness

```
api.MapGet("/production-readiness", async (
    HttpContext context,
    IConfiguration configuration,
    IHostEnvironment environment,
    ISysAssistApiService service,
    IlicenseService licenseService,
    IClock clock,
    CancellationToken cancellationToken) =>
    Results.Ok(await BuildProductionReadinessAsync(
        context, configuration, environment, service,
        licenseService, clock, cancellationToken)))
    .RequireAuthorization("RequireSeniorAdmin")
    .WithTags("Production");
```

2.3 Модель данных CockroachDB и EF Core

CockroachDB используется как production database target. Это PostgreSQL-compatible SQL база, с которой приложение работает через Npgsql и EF Core. Для SysAssist важны транзакционность, индексы, JSONB-поля для payload и возможность хранить структурированные журналы без перехода к отдельному NoSQL-хранилищу.

SysAssistDbContext содержит DbSet для пользователей, ролей, событий, рекомендаций, approval requests, интеграционных модулей, настроек, действий, истории health checks, webhook events, audit entries, system logs, notifications и support bundle exports. Каждая таблица играет роль в цепочке доказуемой эксплуатации.

Миграции EF Core описывают схему, а seeder выполняет idempotent initialization. При первом запуске создаются роли, bootstrap admin, 13 модулей, настройки и действия. В production автоматическое применение миграций отключается, потому что изменение схемы должно проходить через controlled deployment pipeline.

JSONB используется для внешних payload, результатов выполнения и деталей логов. Это позволяет хранить структуру события без потери исходного контекста. Например, webhook payload может быть сохранен и одновременно нормализован в IncidentEvent, чтобы оператор видел человекочитаемую карточку.

Для безопасности support bundle не содержит password hashes, raw module secrets, license tokens, JWT signing keys и encryption keys. Такая санитарная модель важна, потому что support bundle предназначен для аудита и диагностики, но не должен становиться каналом утечки секретов.

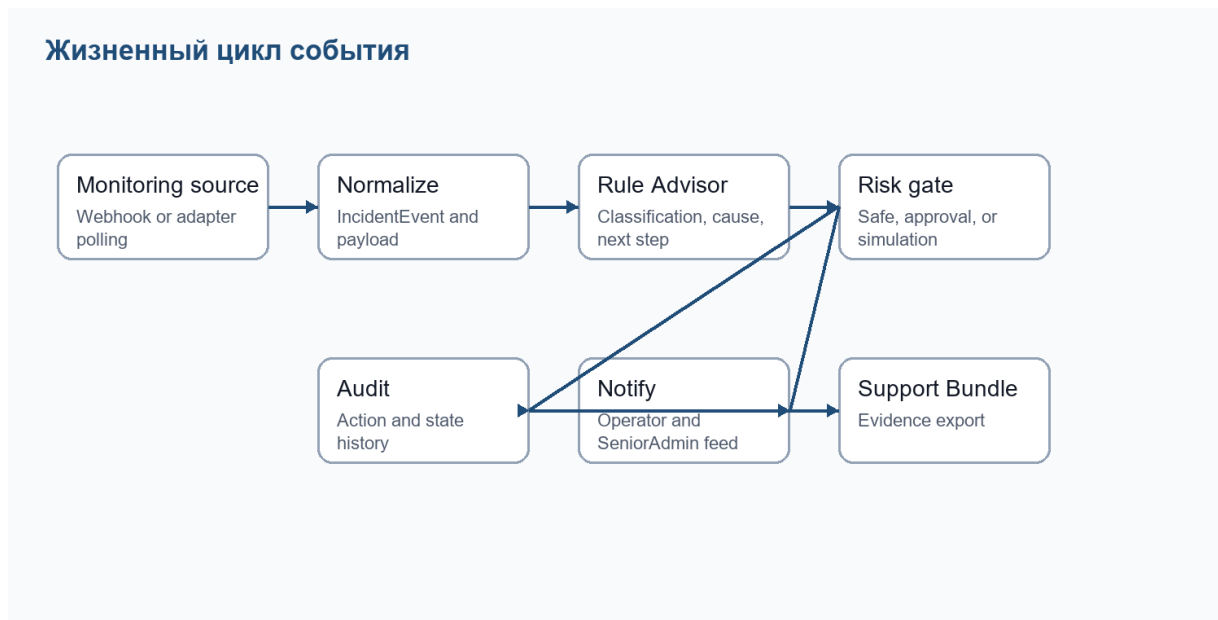


Рисунок 2 - Жизненный цикл события и доказательной базы

Таблица 6 - Ключевые сущности модели данных

Сущность	Назначение	Критичные поля
IncidentEvent	Нормализованное событие/инцидент	source, severity, status, target, payloadJson
EventRecommendation	Рекомендация rule advisor	classification, confidence, probableCause, nextStep
ApprovalRequest	Заявка на опасное действие	status, requestedAt, decidedAt, decisionComment
IntegrationModule	Модуль источника/действия	key, type, isEnabled, healthStatus, safeMode
IntegrationSetting	Настройка модуля	key, value, isSecret, hasValue
AuditEntry	Нефальсифицируемая история операций	actor, action, resource, result, correlationId
SystemLog	Системный журнал	level, component, message, detailsJson

2.4 Реестр модулей и интеграционные адаптеры

Реестр модулей - центральная часть расширяемости SysAssist. Каждый модуль описывается в ModuleCatalog: key, name, type, description, support flags и settings. Общие настройки включают Enabled, UseFallbackMode, SafeMode, PollingEnabled, PollingIntervalSeconds, TimeoutSeconds, RetryCount, Description и Tags.

В baseline-реестр включены 13 модулей: Zabbix, Grafana, Prometheus Alertmanager, PostgreSQL, Redis, Docker, Nginx, Linux Host, HTTP Endpoint Checker, File System Monitor, SMTP Email, Telegram Bot и Local Rule Advisor. Такая композиция покрывает мониторинг, БД, инфраструктуру, уведомления и локальные рекомендации.

Адаптерная модель реализует единый интерфейс для health checks, fetch events и action execution. Это означает, что API не знает внутренних деталей Zabbix, Grafana или Nginx: он вызывает фабрику адаптеров по ключу модуля и получает стандартизированный результат. Благодаря этому workflow событий и аудита остается единым.

Real mode validation предотвращает включение модуля без требуемых настроек. Например, SMTP Email требует Smtphost и FromEmail, Telegram Bot требует BotToken и DefaultChatId, HTTP Endpoint Checker требует EndpointUrl. Если модуль не настроен, результат должен быть NotConfigured, а не фиктивный Healthy.

В текущей рабочей конфигурации включены только необходимые здоровые модули: Grafana, HTTP Endpoint Checker, Local Rule Advisor, Nginx, Prometheus Alertmanager и Zabbix. Неиспользуемые Docker, File System, Linux Host, PostgreSQL, Redis, SMTP Email и Telegram Bot отключены, чтобы dashboard и диагностика не показывали искусственно завышенный счетчик.

Листинг 3 - Фрагмент ModuleCatalog с общими настройками

```
private static readonly SettingDefinition[] CommonSettings =  
[  
    new("Enabled", "bool", DefaultValue: "true"),
```

```

new("UseFallbackMode", "bool", DefaultValue: "false"),
new("SafeMode", "bool", DefaultValue: "true"),
new("PollingEnabled", "bool", DefaultValue: "true"),
new("PollingIntervalSeconds", "int", DefaultValue: "60"),
new("TimeoutSeconds", "int", DefaultValue: "15"),
new("RetryCount", "int", DefaultValue: "3")
];

```

Таблица 7 - Модули SysAssist и эксплуатационный статус

Ключ	Модуль	Тип	Статус
grafana	Grafana	Alerting	Enabled, Healthy
http-endpoint	HTTP Endpoint Checker	LocalCheck	Enabled, Healthy
local-rule-advisor	Local Rule Advisor	Advisor	Enabled, Healthy
nginx	Nginx	Infrastructure	Enabled, Healthy
prometheus-alertmanager	Prometheus Alertmanager	Monitoring	Enabled, Healthy
zabbix	Zabbix	Monitoring	Enabled, Healthy
docker	Docker	Infrastructure	Disabled
file-system	File System Monitor	LocalCheck	Disabled
linux-host	Linux Host	Infrastructure	Disabled
postgresql	PostgreSQL	Database	Disabled
redis	Redis	Database	Disabled
smtp-email	SMTP Email	Notification	Disabled
telegram-bot	Telegram Bot	Notification	Disabled

2.5 Безопасность, секреты и лицензирование

Безопасность SysAssist реализуется несколькими слоями. На транспортном и HTTP-уровне используются HTTPS redirection в не-development окружении, HSTS, security headers, ограничение размера request body и rate limiting. На уровне идентификации применяется JWT Bearer. На уровне полномочий - RBAC policies.

Секреты модулей защищаются через ISecretProtector. Реализация AesGcmSecretProtector добавляет префикс enc:v1, генерирует nonce, шифрует plaintext через AES-GCM и использует associated data SysAssist.IntegrationSetting.Secret.v1. При чтении защищенного значения

выполняется обратная операция, а ошибка дешифрования трактуется как проблема ключа.

В реальном режиме отсутствие ключа Security:SecretEncryptionKey или SYSASSIST_SECRET_ENCRYPTION_KEY приводит к отказу запуска, если RequireEncryptionInRealMode включен. Это правильное поведение: лучше не стартовать, чем хранить реальные module credentials в открытом виде.

Лицензирование построено на проверке ECDSA P-256 подписи. LicenseService анализирует license key, public key, product, validity window и machine fingerprint. Если лицензия невалидна, protected /api/* routes блокируются кодом 402 Payment Required, но auth/license и production-readiness остаются доступными для восстановления.

Важной частью hardening является production-readiness endpoint. Он не раскрывает секреты, но проверяет, что JWT key не dev-only, encryption key задан, bootstrap password сильный, rate limits включены, CORS и AllowedHosts production-ready, база доступна, license valid, diagnostics fresh, enabled modules healthy и SafeMode включен.

Листинг 4 - AES-GCM защита секретов модулей

```
var nonce = RandomNumberGenerator.GetBytes(12);
var plaintext = Encoding.UTF8.GetBytes(value);
var ciphertext = new byte[plaintext.Length];
var tag = new byte[16];

using var aes = new AesGcm(_key, tag.Length);
aes.Encrypt(nonce, plaintext, ciphertext, tag, AssociatedData);

return $"enc:v1:{Convert.ToBase64String(payload)}";
```

Таблица 8 - Production-readiness gates

Gate	Проверяемое условие	Причина
environment-production	ASPNETCORE_ENVIRONMENT=Production	Исключить dev-only поведение
startup-migrations-off	Авто-миграции выключены	Миграции должны идти через pipeline
allowed-hosts-explicit	AllowedHosts не wildcard	Снизить риск host-header атак

cors-production-origins	Только HTTPS frontend origins	Исключить локальные origins в prod
module-secret-encryption	AES-GCM key задан	Защитить module credentials
enabled-modules-healthy	Все enabled модули Healthy	Не выпускать систему с фиктивной связью
fallback-mode-off	Нет fallback у enabled modules	Не маскировать реальные ошибки
safe-mode-on	SafeMode включен	Сдерживать опасные действия

2.6 Frontend-интерфейс оператора

Клиентская часть является рабочей консолью. Основной AppShell содержит sidebar navigation, topbar, поисковую строку, badges состояния, pages switch и toast notifications. В отличие от лендинга, пользователь после входа сразу получает доступ к dashboard и операционным разделам.

Dashboard показывает агрегированные данные: intake, module health, severity distribution, approvals и компактные операционные карточки. Счетчик enabled modules строится по фактическому реестру, а не по статическому числу. Это важно, потому что пользователь ранее видел проблему с ненастоящим счетчиком модулей.

Раздел Modules позволяет управлять подключенными модулями, открывать store, видеть status, real mode, health timestamps, выполнять Save, Test, Connect, Disable и Defaults. UI маскирует секретные настройки и отделяет замену секретов от обычного сохранения settings.

Раздел Diagnostics отображает health score, findings и production gates. После запуска диагностики экран показывает evidence=real-adapter-health, freshness=fresh, healthyModules, enabledModules и warning/blocker counters. Таким образом диагностика перестает быть декоративной и становится эксплуатационным инструментом.

Раздел Logs показывает системные события с деталями. Для login событий записываются component, correlation id и JSON details с

client/login/reason, но без пароля или токена. Support Bundle позволяет выгрузить доказательную базу без секретов, что полезно для сопровождения и аудита.

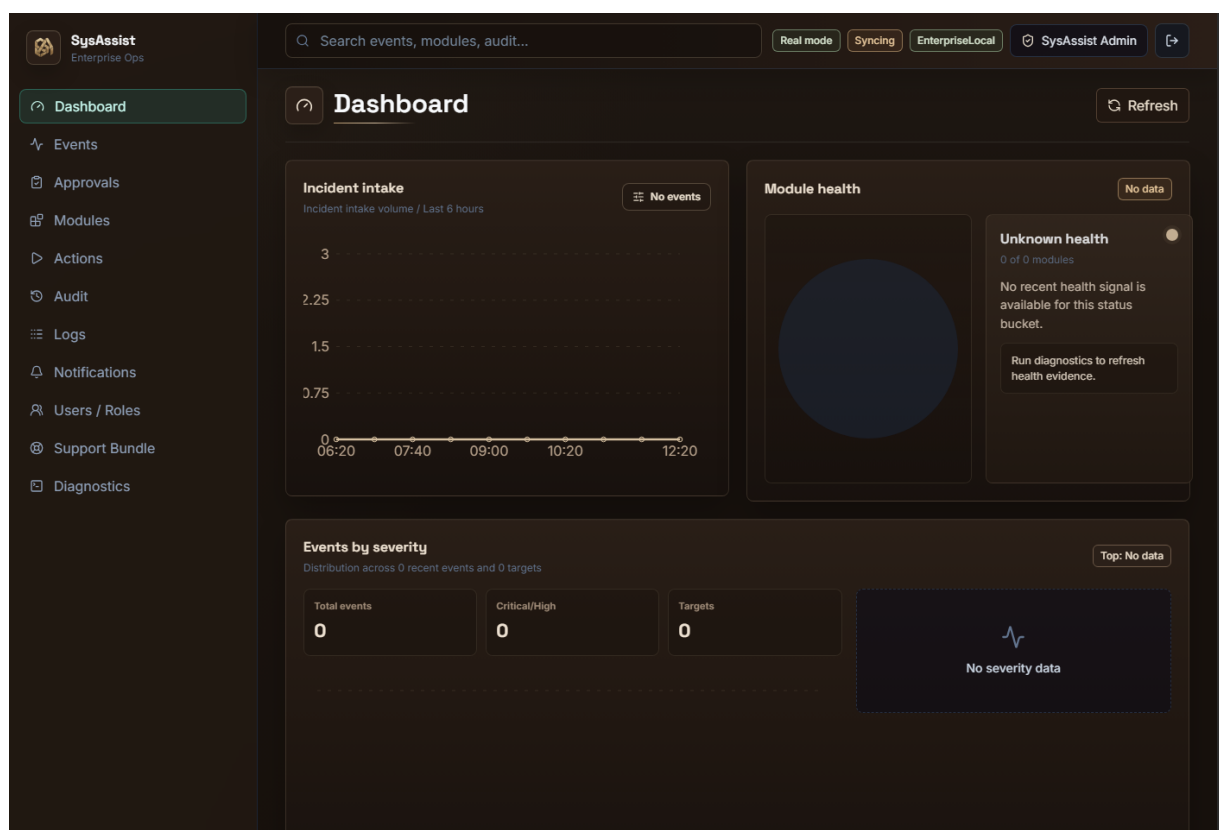


Рисунок 3 - Главная панель SysAssist

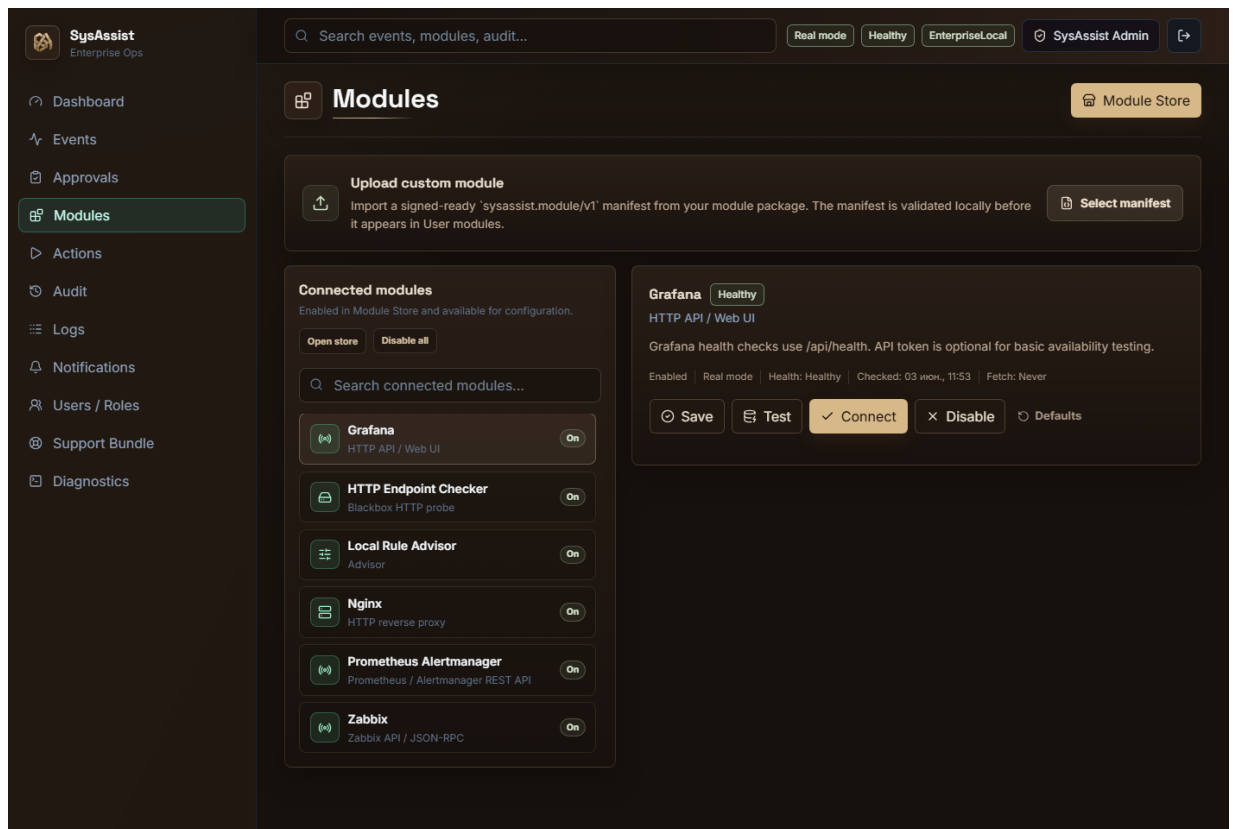


Рисунок 4 - Реестр подключенных модулей

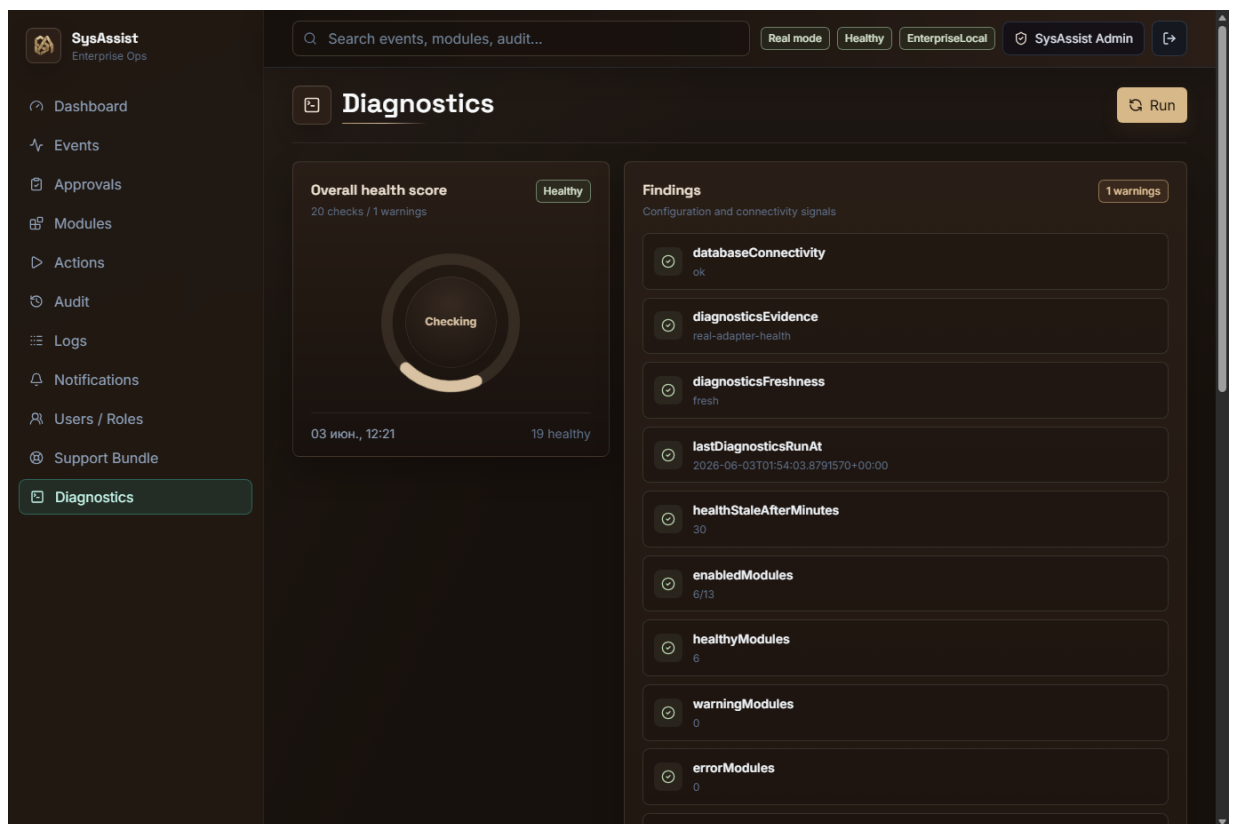


Рисунок 5 - Диагностика на основе real-adapter-health

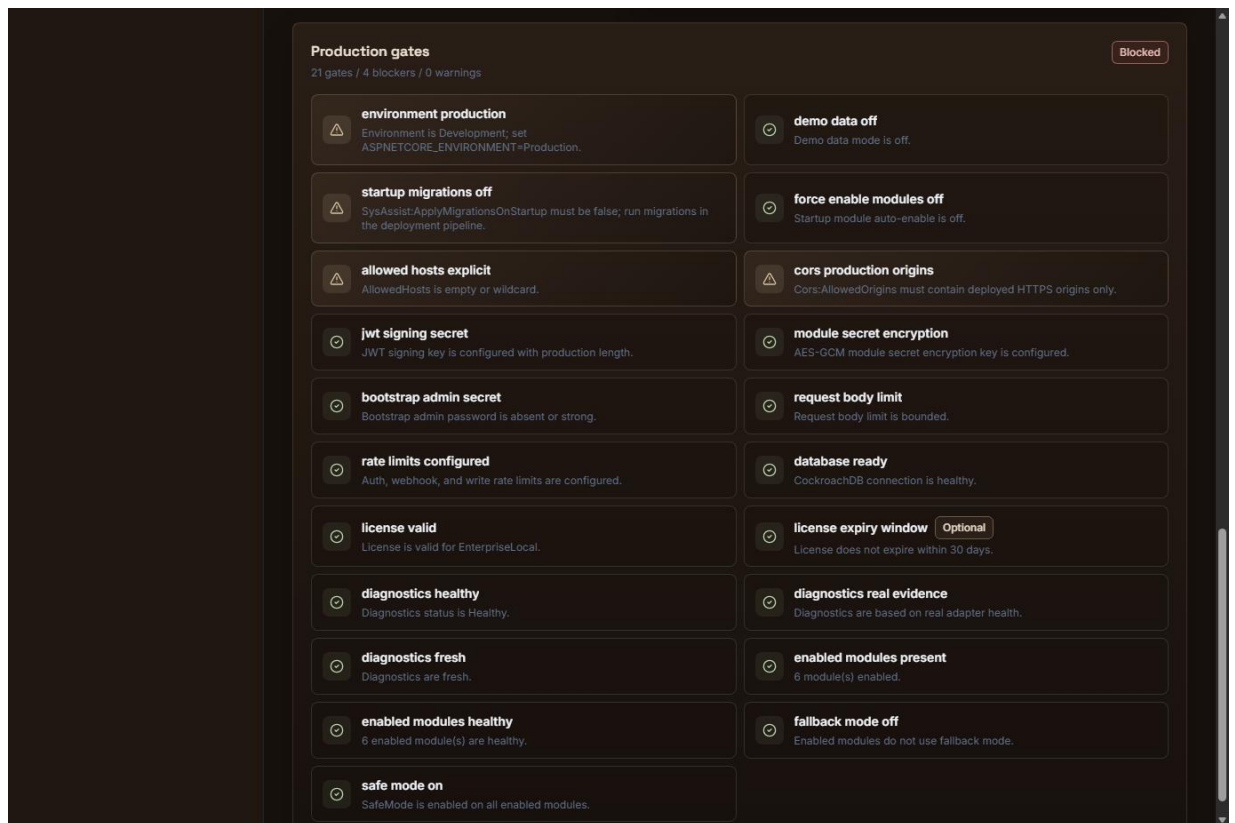


Рисунок 6 - Production-readiness gates

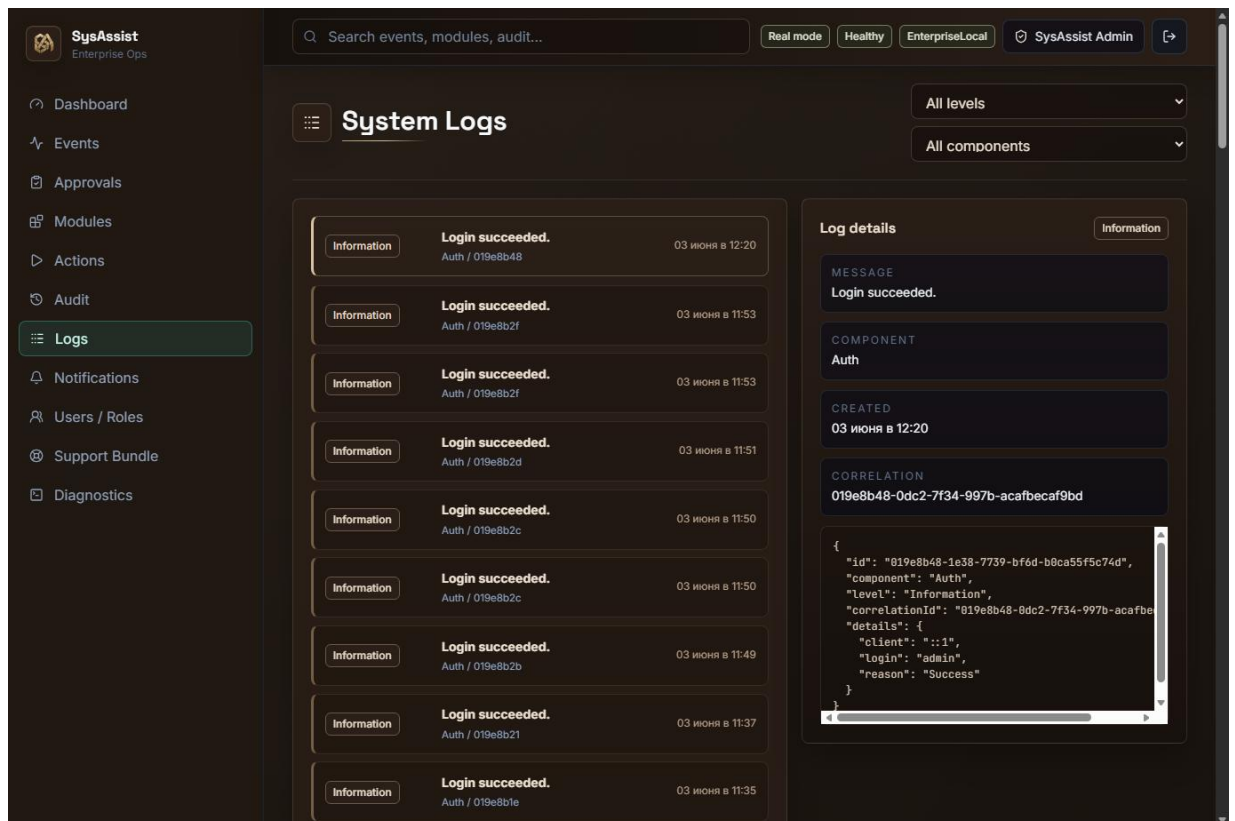


Рисунок 7 - Системные журналы и correlation details

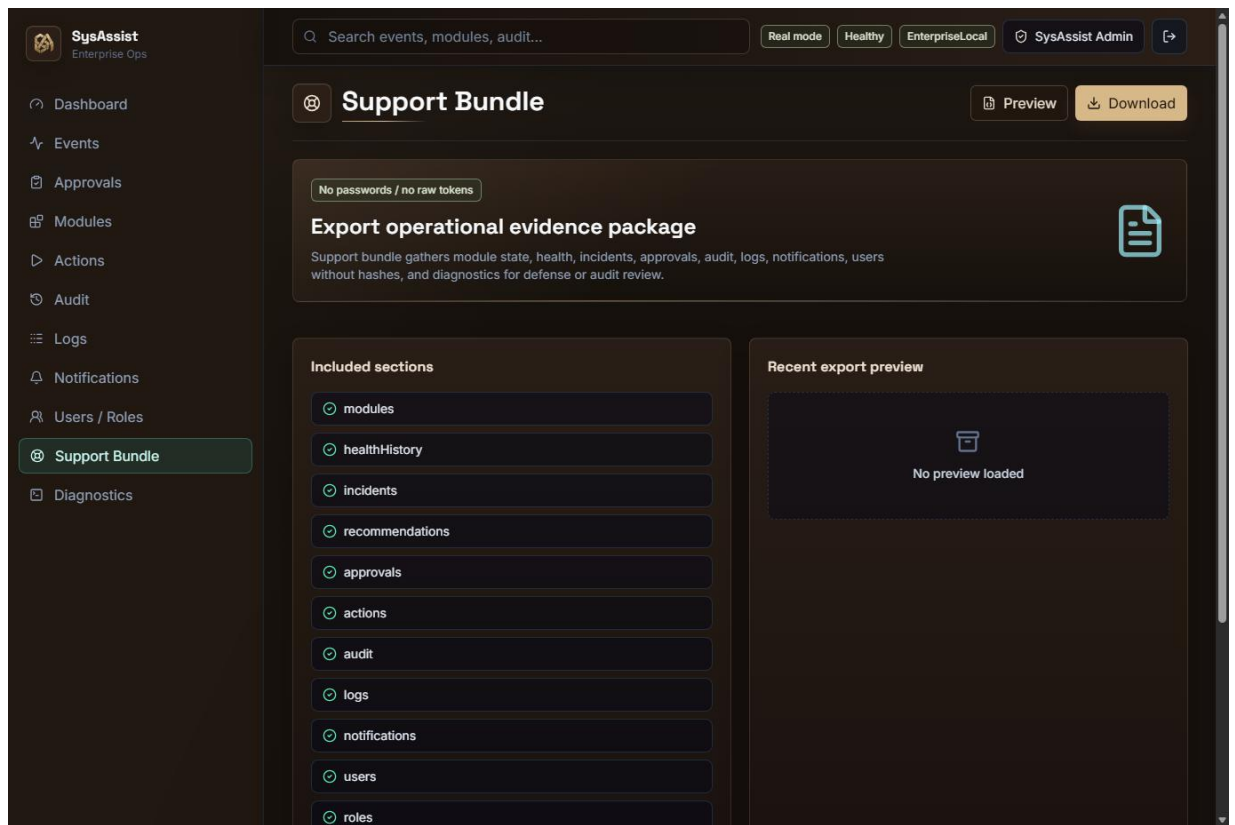


Рисунок 8 - Support Bundle без raw tokens и паролей

3 ТЕСТИРОВАНИЕ, ЭКСПЛУАТАЦИЯ И ОЦЕНКА РЕЗУЛЬТАТА

3.1 Методика тестирования

Тестирование SysAssist выполнено на нескольких уровнях. Unit/integration-like тесты xUnit проверяют password hashing, module state changes, required settings validation, fallback fetch events, event normalization, recommendation creation, high/critical approval creation, approve/reject audit behavior и support bundle security.

Backend test suite запускается командой `dotnet test SysAssist.sln --no-restore`. На момент подготовки диплома результат составил 23 теста из 23 успешно. Это подтверждает, что ключевые сценарии workflow и безопасности проходят автоматическую проверку.

Frontend проверяется через `npm run build`. Сборка выполняет TypeScript build и Vite production build. Дополнительно Vite настроен на code splitting: react-vendor, charts-vendor, motion-vendor, icons-vendor, query-vendor и vendor. Это уменьшает риск избыточного монолитного bundle.

Сквозная smoke-проверка выполняется скриптом `scripts/prod-smoke.ps1`. Он проверяет `/health/ready`, `login`, `/api/auth/me`, `dashboard`, `modules`, `diagnostics/run`, `diagnostics`, `logs` и `support bundle`. Отдельный флаг `-RequireProductionReadiness` заставляет проверку падать, если `production-readiness status` не `Ready`.

Проверка живого API показала `Ready` по `/health/ready`, CockroachDB/Npgsql как provider, Valid license, 6 enabled modules, 7 disabled modules и `Diagnostics=Healthy`. Это подтверждает, что система работает на реальной БД и не опирается на демонстрационный fallback для включенных модулей.

Таблица 9 - Результаты проверок качества

Проверка	Результат	Комментарий
----------	-----------	-------------

dotnet test	23/23 passed	Покрыты auth, modules, workflow, approvals, audit, support bundle
npm run build	passed	TypeScript и Vite production build успешны
/health/ready	Ready	База CockroachDB доступна, лицензия валидна
prod-smoke.ps1	passed	Dashboard counter, module health, diagnostics, logs, support bundle
prod-smoke.ps1 - RequireProductionReadiness	expected block	Локальный dev-конфиг блокируется 4 gate-условиями
secret scan	no real tokens found	Вне .env найдены только имена переменных и код обработки секретов

Листинг 5 - Фрагмент production smoke check

```
$ready = Invoke-Json "$ApiUrl/health/ready"
$loginResponse = Invoke-Json "$ApiUrl/api/auth/login" "POST" @{ } @{
    login = $Login
    password = $adminPassword
}
$modules = As-Array (Invoke-Json "$ApiUrl/api/modules" "GET" $headers)
$diagnosticsRun = Invoke-Json "$ApiUrl/api/diagnostics/run" "POST" $headers
$productionReadiness = Invoke-Json "$ApiUrl/api/production-readiness" "GET" $headers
if ($RequireProductionReadiness -and $productionReadiness.status -ne "Ready") {
    throw "Production readiness blocked"
}
```

3.2 Диагностика, production-readiness и support bundle

Диагностика SysAssist строится на данных, полученных из модулей и базы. В составе findings присутствуют databaseConnectivity, diagnosticsEvidence, diagnosticsFreshness, lastDiagnosticsRunAt, healthStaleAfterMinutes, enabledModules, healthyModules, warningModules, errorModules и другие компоненты. Warning формируется только при реальной проблеме: stale health, missing secret protection, invalid license, not configured modules или errors.

Production-readiness является более строгой проверкой, чем diagnostics. Diagnostics отвечает на вопрос, здорова ли система сейчас. Production-readiness отвечает на вопрос, можно ли выкатывать текущую конфигурацию как production. Именно поэтому локальный Development может иметь Healthy diagnostics, но Blocked production readiness.

В текущем локальном запуске production-readiness показывает 21 gate, 4 required blockers и 0 optional warnings. Блокеры: environment-production, startup-migrations-off, allowed-hosts-explicit и cors-production-origins. Это ожидаемый результат, потому что локальная среда разработки не должна маскироваться под production.

Support Bundle формирует JSON-доказательную базу. В него входят modules, healthHistory, incidents, recommendations, approvals, actions, audit, logs, notifications, users, roles и diagnostics. При этом raw secrets, password hashes, JWT signing key, encryption key и license tokens не включаются.

Такая связка diagnostics + production-readiness + support bundle делает систему пригодной для сопровождения. Оператор видит текущую картину, инженер запускает диагностику, администратор видит релизные blockers, аудитор выгружает доказательства без доступа к секретам.

Таблица 10 - Smoke output текущей конфигурации

Показатель	Значение
Ready	Ready
User	admin
EnabledModules	6
DisabledModules	7
Diagnostics	Healthy
ProductionReadiness	Blocked
ProductionBlockers	4
DiagnosticsRun	Diagnostics ran 6 real adapter checks. Healthy: 6; attention: 0.
SupportProduct	SysAssist

3.3 Развертывание и сопровождение

Развертывание SysAssist включает backend API, frontend static build и CockroachDB. Локальный Development может применять миграции при старте, но production должен выполнять миграции отдельным шагом

deployment pipeline. Это предотвращает неконтролируемое изменение схемы при каждом старте процесса.

Для production запуска должны быть заданы секреты: connection string базы, JWT signing key, SecretEncryptionKey, license public key, license key и bootstrap admin password для свежей БД. Секреты должны поступать из secret storage, а не храниться в исходном коде. Файл .env допустим только для локального контура и исключается из публикации.

AllowedHosts должен быть явным, а Cors:AllowedOrigins должен содержать только deployed HTTPS frontend origins. Локальные localhost/127.0.0.1 origins являются development-условием и должны блокировать production-readiness. Такой контроль снижает риск случайного выпуска dev-конфигурации.

Сопровождение включает мониторинг логов, регулярный запуск diagnostics, анализ production gates, выгрузки support bundle при инциденте, ротацию секретов и проверку license expiry. Для регулируемых сред желательно добавить внешний immutable audit sink, SSO/OIDC и централизованный secrets manager.

Фронтенд может быть развернут как статический build через nginx или managed static hosting, backend - как сервис за reverse proxy/ingress. При интернет-экспозиции необходимы WAF/IAP, строгий HTTPS, лимиты запросов и аудит login failures.

Таблица 11 - Production deployment checklist

Шаг	Критерий приемки
1	ASPNETCORE_ENVIRONMENT=Production
2	SysAssist:UseDemoData=false
3	SysAssist:ApplyMigrationsOnStartup=false
4	ConnectionStrings:SysAssistDb указывает на CockroachDB
5	Jwt:SigningKey/Auth:JwtSecret задан как secret длиной >= 32
6	Security:SecretEncryptionKey задан для AES-GCM

7	Licensing:PublicKey и Licensing:LicenseKey установлены
8	AllowedHosts и Cors:AllowedOrigins не содержат wildcard/localhost
9	/health/ready возвращает Ready
10	scripts/prod-smoke.ps1 -RequireProductionReadiness проходит без blockers

3.4 Экономическая и практическая оценка

Практический эффект SysAssist проявляется в сокращении времени между обнаружением события и принятием решения. Если раньше оператору нужно было вручную сопоставлять источник, искать регламент, согласовывать действие и фиксировать результат, то SysAssist формирует единую карточку инцидента, рекомендацию, approval request и audit trail.

Экономический эффект можно оценивать через снижение трудозатрат на рутинные проверки, уменьшение повторяющихся ошибок и сокращение времени подготовки отчетности. Support bundle снимает часть нагрузки с инженера при расследовании, потому что собирает состояние модулей, health history, incidents, recommendations, approvals, actions, audit, logs, notifications, users и diagnostics.

Технический эффект выражается в повышении воспроизводимости. Каждое действие связано с пользователем, correlation id, модулем, статусом и результатом. Система фиксирует, было ли действие исполнено или симулировано SafeMode. Это особенно важно при разборе критических инцидентов.

Ограничения проекта также зафиксированы. SSO/OIDC и refresh-token lifecycle не реализованы, внешний immutable audit sink отсутствует, provider-specific webhook signatures требуют дальнейшего развития, background polling должен быть оформлен как hosted scheduler with leases and retry policy. Эти ограничения не снижают ценность дипломной реализации, но задают roadmap.

В целом SysAssist доведен до состояния продуктовой основы: есть реальная БД, ролевой доступ, модульный реестр, диагностика, gates, лицензирование, шифрование секретов, поддержка smoke-тестирования, UI и доказательная выгрузка. Для промышленного внедрения требуется закрыть интеграционные и организационные пункты roadmap.

ЗАКЛЮЧЕНИЕ

В результате дипломного проекта разработана и описана платформа SysAssist - веб-приложение для автоматизации обработки ИТ-инцидентов и управления регламентированными действиями в инфраструктуре предприятия. Система объединяет мониторинговые источники, модульный реестр, workflow согласований, аудит, диагностику и support bundle.

В аналитической части рассмотрены проблемы разрыва между мониторингом и действиями, ограничения ручной координации и необходимость формального workflow. Определены требования к системе: модульность, ролевой доступ, реальная диагностика, защита секретов, лицензирование и production-readiness.

В проектно-технической части обоснован стек .NET 9, ASP.NET Core, React, TypeScript, Vite, CockroachDB, EF Core/Npgsql, Serilog, JWT, AES-GCM и ECDSA licensing. Описаны Clean Architecture, модель данных, реестр 13 модулей, adapter runtime, API routes, frontend pages и security hardening.

В практической части выполнены тестирование и эксплуатационная проверка. Backend test suite прошел 23/23 тестов, frontend production build выполнен успешно, health/ready вернул Ready, diagnostics показала Healthy и real-adapter-health, smoke script подтвердил согласованность dashboard counter, module health, logs и support bundle.

Особым результатом является production-readiness endpoint. Он делает качество системы проверяемым: локальный Development честно получает Blocked, а production должен пройти обязательные gates. Это приближает проект к продуктовой инженерии, где релиз определяется не визуальной готовностью интерфейса, а набором проверяемых условий.

Разработанная система может быть использована как основа для дальнейшего внедрения в корпоративный контур. Перспективы развития включают SSO/OIDC, refresh tokens, immutable audit storage, расширенные

webhook signatures, hosted scheduler, backup/restore automation и более глубокие provider-specific actions.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Microsoft Learn. ASP.NET Core documentation. - URL: <https://learn.microsoft.com/aspnet/core/>.
2. Microsoft Learn. Entity Framework Core documentation. - URL: <https://learn.microsoft.com/ef/core/>.
3. Microsoft Learn. Authentication and authorization in ASP.NET Core. - URL: <https://learn.microsoft.com/aspnet/core/security/>.
4. Cockroach Labs. CockroachDB documentation. - URL: <https://www.cockroachlabs.com/docs/>.
5. Npgsql Documentation. PostgreSQL provider for .NET. - URL: <https://www.npgsql.org/>.
6. React documentation. - URL: <https://react.dev/>.
7. Vite documentation. - URL: <https://vite.dev/>.
8. TypeScript documentation. - URL: <https://www.typescriptlang.org/docs/>.
9. OWASP Application Security Verification Standard. - URL: <https://owasp.org/www-project-application-security-verification-standard/>.
10. OWASP Cheat Sheet Series. Authentication, secrets management and logging guidance. - URL: <https://cheatsheetseries.owasp.org/>.
11. Serilog documentation. - URL: <https://serilog.net/>.
12. JSON Web Token RFC 7519. - URL: <https://www.rfc-editor.org/rfc/rfc7519>.
13. NIST. Guide to Integrating Forensic Techniques into Incident Response. SP 800-86.
14. ГОСТ 34.602-2020. Информационная технология. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы.
15. ГОСТ 19.701-90. Единая система программной документации. Схемы алгоритмов, программ, данных и систем.
16. Федеральный закон Российской Федерации от 27.07.2006 № 152-ФЗ «О персональных данных».
17. Документация проекта SysAssist: README.md, docs/architecture.md, docs/prod-readiness.md.
18. Исходный код проекта SysAssist: локальный репозиторий C:/Users/8-Bits/Desktop/Sys/SysAssist.

ПРИЛОЖЕНИЕ А. API-КОНТРАКТЫ И DTO

В приложении приведены фрагменты контрактов, используемых frontend-клиентом и backend API. Контракты отделены от EF-сущностей, что снижает риск передачи лишних внутренних полей во внешний HTTP surface.

Листинг A.1 - DTO модуля и настройки

```
public sealed record ModuleDto(
    Guid Id,
    string Key,
    string Name,
    string Type,
    string? Description,
    bool IsEnabled,
    string HealthStatus,
    DateTimeOffset? LastHealthCheckAt,
    DateTimeOffset? LastFetchAt,
    bool SupportsPolling,
    bool SupportsWebhooks,
    bool SupportsActions,
    bool UseFallbackMode,
    bool SafeMode);

public sealed record ModuleSettingDto(
    Guid Id,
    string Key,
    string? Value,
    bool IsSecret,
    bool IsRequired,
    string ValueType,
    string? Description,
    bool HasValue);
```

Листинг A.2 - DTO события, рекомендации и approval

```
public sealed record EventDto(
    Guid Id,
    string? ExternalEventId,
    DateTimeOffset CreatedAt,
    string Source,
    string EventType,
    string Severity,
    string Target,
    string Status,
    string? CorrelationId,
    string Summary,
    Guid? ModuleId,
    string? PayloadJson,
    EventRecommendationDto? Recommendation);

public sealed record ApprovalDto(
    Guid Id,
    Guid EventId,
    Guid ActionId,
    string Status,
    DateTimeOffset RequestedAt,
    Guid? RequestedByUserId,
    DateTimeOffset? DecidedAt,
    Guid? DecidedByUserId,
    string? DecisionComment);
```

ПРИЛОЖЕНИЕ Б. PRODUCTION-READINESS МАТРИЦА

Матрица используется как автоматизированный release gate. Для локальной разработки допустим статус Blocked, но для промышленного запуска скрипт prod-smoke.ps1 должен проходить с ключом - RequireProductionReadiness.

Таблица Б.1 - Матрица production gates

Gate	Тип	Текущий статус	Действие
environment-production	Required	Blocked locally	Set ASPNETCORE_ENVIRONMENT=Production
demo-data-off	Required	Passed	Keep SysAssist:UseDemoData=false
startup-migrations-off	Required	Blocked locally	Disable startup migrations
force-enable-modules-off	Required	Passed	Do not force-enable modules on startup
allowed-hosts-explicit	Required	Blocked locally	Set explicit AllowedHosts
cors-production-origins	Required	Blocked locally	Use HTTPS deployed frontend origins
jwt-signing-secret	Required	Passed	Secret length >= 32 and not dev-only
module-secret-encryption	Required	Passed	AES-GCM encryption key configured
database-ready	Required	Passed	CockroachDB connection healthy
license-valid	Required	Passed	EnterpriseLocal license valid
diagnostics-real-evidence	Required	Passed	real-adapter-health
enabled-modules-healthy	Required	Passed	6 enabled modules healthy

ПРИЛОЖЕНИЕ В. ЭКСПЛУАТАЦИОННЫЕ КОМАНДЫ

Листинг В.1 - Команды сборки и проверки

```
dotnet restore SysAssist.sln
dotnet build SysAssist.sln
dotnet test SysAssist.sln --no-restore

cd src/SysAssist.Web
npm run build
```

```
cd C:\Users\8-Bits\Desktop\Sys\SysAssist
scripts/prod-smoke.ps1
scripts/prod-smoke.ps1 -RequireProductionReadiness
```

Листинг В.2 - Проверка API после запуска

```
Invoke-RestMethod http://localhost:5089/health/ready

$body = @{
    login = "admin"
    password = $env:SYSASSIST_BOOTSTRAP_ADMIN_PASSWORD
} | ConvertTo-Json
$login = Invoke-RestMethod http://localhost:5089/api/auth/login `
    -Method Post -ContentType "application/json" -Body $body
$headers = @{ Authorization = "Bearer $($login.accessToken)" }
Invoke-RestMethod http://localhost:5089/api/production-readiness -Headers $headers
```

ПРИЛОЖЕНИЕ Г. ДОРОЖНАЯ КАРТА РАЗВИТИЯ

Таблица Г.1 - Roadmap продукта

Направление	Содержание
SSO/OIDC	Подключить корпоративного identity provider и заменить локальный login в production
Refresh token lifecycle	Добавить refresh tokens, revoke list, device sessions и rotation
Immutable audit sink	Дублировать audit entries во внешний журнал с защитой от изменения
Hosted polling scheduler	Вынести polling в hosted service с leases, retries и backoff
Webhook signatures	Реализовать provider-specific signature validation для Zabbix/Grafana/Alertmanager
Backup/restore drill	Автоматизировать резервное копирование CockroachDB и проверку восстановления
Secrets rotation	Реализовать re-encryption job перед сменой SecretEncryptionKey
Provider actions	Расширить безопасные действия по модулям: nginx config test, docker restart, Redis info, PostgreSQL checks

ПРИЛОЖЕНИЕ Д. ТЕХНИЧЕСКОЕ ЗАДАНИЕ НА SYSASSIST

Д.1 Назначение и область применения

SysAssist предназначен для автоматизации координации ИТ-инцидентов, поступающих из систем мониторинга и инфраструктурных источников. Система применяется в контуре сопровождения программного обеспечения и инфраструктуры предприятия, где требуется связать событие, рекомендацию, регламентированное действие, согласование и доказательную запись.

Область применения включает дежурные смены, инженерные команды сопровождения, администраторов платформы и аудиторов. Для оператора система является точкой входа в текущую картину инцидентов, для инженера - инструментом проверки модулей и запуска диагностики, для администратора - средством настройки модулей и пользователей, для аудитора - источником support bundle.

Система не является заменой Zabbix, Grafana, Prometheus Alertmanager, CockroachDB, Redis, Docker, Nginx, SMTP или Telegram. Она выполняет роль координационного слоя, который читает сигналы и организует безопасную обработку. Такое ограничение важно зафиксировать в ТЗ, чтобы не смешивать функции мониторинга, ITSM и низкоуровневого управления инфраструктурой.

Д.2 Функциональные требования

Система должна обеспечивать вход пользователя по логину и паролю с выдачей JWT access token. После входа пользователь должен видеть только те разделы, которые разрешены его ролями. Неавторизованный доступ к protected routes должен блокироваться.

Система должна отображать dashboard с фактическими счетчиками: количество активных событий, pending approvals, enabled modules и health

summary. Счетчики должны строиться на данных реестра и базы, а не быть зашитыми в frontend.

Система должна хранить реестр модулей, включающий key, name, type, description, enabled state, health status, last health check, last fetch, polling/webhook/action capabilities, fallback mode и SafeMode. Для каждого модуля должны быть доступны настройки, secret replacement, test connection, health check и fetch events при наличии соответствующей capability.

Система должна принимать webhook события от Zabbix, Grafana и Prometheus Alertmanager, сохранять raw payload, нормализовать incident event и связывать его с recommendation. При high/critical risk должна создаваться approval request.

Система должна предоставлять audit trail и system logs. Audit фиксирует бизнес-действия, system logs фиксируют технические события. Оба журнала должны поддерживать correlation id для сквозной трассировки.

Система должна формировать support bundle в JSON-формате. Выгрузка должна включать состояние модулей, health history, incidents, recommendations, approvals, actions, audit, logs, notifications, users, roles и diagnostics, но не должна включать password hashes, raw secrets и signing/encryption keys.

Таблица Д.1 - Функциональные требования и критерии приемки

ID	Требование	Критерий приемки
FR-01	Авторизация пользователя	POST /api/auth/login возвращает токен и CurrentUserDto
FR-02	Dashboard	EnabledModulesCount равен числу включенных модулей в /api/modules
FR-03	Module registry	13 baseline modules доступны через /api/modules
FR-04	Secret masking	Secret settings возвращают ***** и hasValue=true
FR-05	Diagnostics	GET /api/diagnostics содержит diagnosticsEvidence=real-adapter-health

FR-06	Production gates	GET /api/production-readiness возвращает gates и статус Ready/Blocked
FR-07	Support bundle	JSON export не содержит password hashes и raw secrets
FR-08	Audit	Мутации модулей и approval decisions пишутся в audit
FR-09	Rate limiting	Auth/webhook/write endpoints защищены лимитами
FR-10	License wall	Protected /api routes блокируются при невалидной лицензии

Д.3 Нефункциональные требования

Система должна запускаться локально для разработки и демонстрации, но иметь отдельную модель production readiness. Локальный Development не должен считаться production-ready, если включены startup migrations, wildcard AllowedHosts или localhost CORS.

Система должна обеспечивать защищенное хранение module secrets. В real mode наличие encryption key является обязательным. Секреты не должны выводиться в UI, logs, support bundle или smoke output.

Система должна иметь контролируемую модель отказов. Недоступность внешнего модуля не должна приводить к падению API: адаптер возвращает NotConfigured, Warning или Error, а диагностика отражает фактическое состояние.

Система должна быть расширяемой. Новый модуль должен добавляться через ModuleCatalog, settings definition и adapter implementation без изменения общего workflow событий и approvals.

Система должна быть проверяемой. Минимальный набор проверок включает unit/integration tests, frontend build, /health/ready, diagnostics, production-readiness и prod-smoke.ps1.

ПРИЛОЖЕНИЕ Е. ПРОГРАММА И МЕТОДИКА ИСПЫТАНИЙ

Программа испытаний описывает набор проверок, достаточный для приемки учебной версии продукта. Испытания разделены на backend, frontend, безопасность, модули, диагностику и эксплуатационные сценарии. Каждая проверка имеет ожидаемый результат, который должен быть воспроизводимым на чистом запуске.

Таблица Е.1 - Набор испытаний SysAssist

ID	Область	Действие	Ожидаемый результат
TC-01	Backend	Запустить dotnet test SysAssist.sln -no-restore	Все тесты passed
TC-02	Backend	Проверить password hashing	Верный пароль проходит, неверный отклоняется
TC-03	Modules	Включить модуль без required settings в real mode	Операция блокируется validation error
TC-04	Modules	Запустить health check включенного Grafana	HealthStatus=Healthy
TC-05	Modules	Отключить неиспользуемый модуль	Dashboard counter уменьшается
TC-06	Security	Запросить secret setting	Value=*****, HasValue=true
TC-07	Security	Заменить secret через dedicated endpoint	Audit содержит MODULE_SETTINGS_UPDATED/secret update
TC-08	Auth	Войти с неправильным паролем	401 и Warning log без password
TC-09	RBAC	Пользователь без роли Admin открывает /api/users	403 Forbidden
TC-10	Diagnostics	POST /api/diagnostics/run	Сообщение о real adapter checks
TC-11	Readiness	GET /health/ready	Ready при доступной БД и валидной лицензии
TC-12	Prod gates	GET /api/production-readiness	Список gates без раскрытия секретов
TC-13	Prod smoke	scripts/prod-smoke.ps1	Проверка завершается JSON output
TC-14	Prod smoke	scripts/prod-smoke.ps1 -	Ожидаемый block по dev gates

		RequireProductionReadiness в Development	
TC-15	Support	GET /api/support-bundle	JSON содержит evidence sections
TC-16	Support	Просканировать bundle	Нет password hashes/raw secrets
TC-17	Frontend	npm run build	TypeScript/Vite build passed
TC-18	Frontend	Открыть Dashboard	Нет login page, счетчики и badges загружены
TC-19	Logs	Открыть System Logs	Login succeeded содержит detailsJson без пароля
TC-20	License	Проверить /api/license	Status=Valid, Edition=EnterpriseLocal

Е.1 Методика выполнения smoke-проверки

Перед запуском smoke-проверки API должен быть поднят на <http://localhost:5089>, а frontend при необходимости - на <http://localhost:5173>. Пароль администратора берется из SYSASSIST_SMOKE_ADMIN_PASSWORD, SysAssist__BootstrapAdminPassword или SYSASSIST_BOOTSTRAP_ADMIN_PASSWORD. Скрипт не выводит значение секрета.

Скрипт сначала вызывает /health/ready. Если база недоступна или лицензия невалидна, проверка останавливается. Далее выполняется login, запрашиваются данные пользователя, dashboard, список модулей, запускается диагностика, читаются logs, support bundle и production-readiness.

Скрипт сверяет dashboard.enabledModulesCount с фактическим количеством enabled modules. Это предотвращает ситуацию, когда интерфейс показывает ненастоящий счетчик. Затем проверяется, что diagnostics.status=Healthy и что все enabled modules имеют HealthStatus=Healthy.

При использовании ключа -RequireProductionReadiness скрипт дополнительно требует productionReadiness.status=Ready. В локальном

Development ожидаемым результатом является блокировка, потому что environment-production, startup-migrations-off, allowed-hosts-explicit и cors-production-origins не проходят.

Листинг E.1 - Пример итогового smoke output

```
{
  "Ready": "Ready",
  "User": "admin",
  "EnabledModules": 6,
  "DisabledModules": 7,
  "Diagnostics": "Healthy",
  "ProductionReadiness": "Blocked",
  "ProductionBlockers": 4,
  "DiagnosticsRun": "Diagnostics ran 6 real adapter checks. Healthy: 6; attention: 0.",
  "SupportProduct": "SysAssist"
}
```

ПРИЛОЖЕНИЕ Ж. РУКОВОДСТВО ОПЕРАТОРА

Ж.1 Вход и первичная навигация

Оператор открывает frontend SysAssist и выполняет вход. После успешной авторизации интерфейс показывает sidebar и dashboard. В верхней панели отображаются badges: Real mode, состояние диагностики/синхронизации, edition лицензии и текущий пользователь.

Dashboard предназначен для быстрого обзора. Оператор должен проверить, что модульные счетчики соответствуют реальному состоянию, а статус health не показывает ошибку. Если статус Syncing или Unknown не меняется, необходимо перейти в Diagnostics и выполнить Run.

Раздел Events используется для просмотра инцидентов, созданных из webhook или adapter fetch. Оператор анализирует source, severity, target, status, correlation id и recommendation. При наличии PendingApproval оператор не выполняет действие самостоятельно, а ожидает решения ответственной роли.

Раздел Notifications показывает локальную ленту уведомлений. Уведомления помогают понять, какие события требуют реакции и кто уже принял решение. Mark read фиксирует обработку уведомления на уровне пользователя.

Ж.2 Работа с инцидентом

При открытии инцидента оператор должен проверить Summary, Payload JSON и Recommendation. Payload сохраняет исходный контекст события, а Recommendation содержит классификацию, confidence, probable cause, next step и suggested action.

Если действие относится к low/medium risk, система может выполнить диагностическую операцию или safe simulation. Если действие high/critical, создается approval request. Это поведение защищает инфраструктуру от несанкционированных изменений.

Оператор не должен обходить approval workflow через прямое выполнение команд вне SysAssist, если событие уже заведено в систему. Иначе audit trail потеряет полноту, а support bundle не сможет доказать цепочку обработки.

После завершения обработки оператор проверяет статус события. Completed, Executed, Simulated, Rejected и PendingApproval должны использоваться последовательно и не противоречить записи в audit.

Ж.3 Проверка доказательной базы

Если требуется передать информацию инженеру или аудитору, оператор открывает Support Bundle. Раздел показывает включенные секции и позволяет скачать JSON export. Перед передачей важно помнить, что bundle не содержит raw tokens и password hashes, но содержит достаточно технических данных для анализа.

При спорной ситуации оператор открывает Logs и Audit. Logs отвечают на вопрос, что произошло технически, Audit - кто выполнил бизнес-действие. Correlation id позволяет связать записи между собой.

Оператор должен избегать изменения настроек модулей без роли и регламента. Настройки подключений, секреты и SafeMode относятся к зоне ответственности инженера или администратора.

ПРИЛОЖЕНИЕ II. РУКОВОДСТВО АДМИНИСТРАТОРА

II.1 Управление модулями

Администратор отвечает за включение только тех модулей, которые действительно используются и имеют параметры подключения. Включение всех 13 модулей без реальной связи создает шум и делает диагностику недостоверной. Поэтому текущая конфигурация содержит 6 enabled healthy modules и 7 disabled modules.

Перед переводом модуля в real mode необходимо заполнить required settings, заменить secret values через dedicated secret endpoint, оставить SafeMode=true и выполнить Test Connection. Только после устойчивого Healthy состояния можно рассматривать дальнейшие действия.

UseFallbackMode должен быть выключен для включенных модулей в продуктовой конфигурации. Fallback допустим для демонстрации, но production-readiness gate fallback-mode-off должен проходить.

SafeMode должен оставаться включенным по умолчанию. Отключение SafeMode допустимо только для заранее проверенных provider actions, после approval и при наличии rollback plan.

Таблица II.1 - Административные правила модулей

Правило	Описание
Enable only needed	Не включать модули, для которых нет рабочей интеграции
Secrets via secret endpoint	Не передавать secret values через обычное сохранение настроек
Test before use	Каждый включенный модуль должен пройти Test/Health
SafeMode by default	Опасные действия симулируются до явного разрешения
No fallback in prod	Fallback mode не должен маскировать отсутствие связи
Audit every mutation	Изменения настроек и статуса модуля фиксируются в audit

II.2 Управление безопасностью

Администратор должен обеспечить хранение .env вне репозитория и не выводить секреты в терминал, документы или support bundle. Проверка

секретов выполняется `grep/rg`-сканом по репозиторию с исключением `.env`, `bin`, `obj`, `dist` и `node_modules`.

JWT signing key должен быть сильным и задаваться через секреты окружения. В production запрещается dev-only fallback. При компрометации ключа необходимо ротировать ключ, инвалидировать старые токены и проверить login logs.

SecretEncryptionKey должен быть задан до записи реальных module secrets. Ротация ключа требует отдельного re-encryption job: нельзя просто заменить ключ, если в базе уже есть enc:v1 payload, потому что старые секреты перестанут расшифровываться.

Лицензия должна контролироваться до запуска protected сценариев. При невалидной лицензии администратор может открыть `/api/license` и `/api/production-readiness`, чтобы увидеть причину блокировки и fingerprint.

Листинг И.1 - Безопасный secret scan без вывода .env

```
rg -n --hidden `
--glob '!.env*' `
--glob '!**/bin/**' `
--glob '!**/obj/**' `
--glob '!**/node_modules/**' `
--glob '!**/dist/**' `
"(sk-[A-Za-z0-9_-]{20,}|ghp_[A-Za-z0-9_-]{20,}|BEGIN (RSA|EC|OPENSSH|PRIVATE) KEY)" .
```

ПРИЛОЖЕНИЕ К. МАТРИЦА УГРОЗ И КОНТРОЛЕЙ

Матрица угроз фиксирует основные риски, актуальные для SysAssist как системы координации ИТ-операций. Для каждой угрозы определен контроль, реализованный в проекте или вынесенный в roadmap.

Таблица К.1 - Матрица угроз SysAssist

Угроза	Контроль	Статус
Утечка module secrets	Secret masking, AES-GCM, support bundle sanitization	Реализовано
Dev-конфиг в production	Production-readiness gates	Реализовано
Подмена host/origin	AllowedHosts explicit, HTTPS CORS origins	Реализовано как gate
Bruteforce login	Auth rate limiting, login security logs	Реализовано
Неавторизованное действие	JWT, RBAC, approval workflow	Реализовано
Опасное действие без проверки	SafeMode=true, risk level, approvals	Реализовано
Фиктивная диагностика	diagnosticsEvidence=real-adapter-health	Реализовано
Утечка через support bundle	No raw secrets/password hashes/tokens	Реализовано
Недостоверный audit	Correlation id and audit entries	Частично; immutable sink в roadmap
Компрометация JWT key	Strong secret, rotation procedure	Частично; revoke/refresh roadmap
Replay webhook	Webhook signature validation	Частично; provider-specific roadmap
Сбой БД	health/ready, backup/restore requirement	Частично; drill roadmap

К.1 Пояснение по residual risk

Даже при реализованных контролях система сохраняет residual risk. Например, RBAC защищает API, но не заменяет корпоративный SSO и lifecycle refresh tokens. AES-GCM защищает секреты в БД, но требует дисциплины ротации ключей и резервного копирования.

Production-readiness снижает риск случайного выпуска Development конфигурации, но не заменяет инфраструктурный контроль: reverse proxy,

HTTPS certificates, WAF/IAP, centralized logging и secret manager должны быть реализованы в deployment environment.

Audit entries полезны для расследования, однако для регулируемых сред их нужно дублировать во внешний immutable sink. Иначе администратор базы теоретически может изменить историю. Этот пункт вынесен в roadmap как обязательный для банковского уровня.

Webhook endpoints должны иметь provider-specific signature validation. Текущая архитектура уже содержит место для проверки подписи, но каждый поставщик имеет свои схемы HMAC/secret/token. Для production требуется реализовать конкретные правила для Zabbix, Grafana и Alertmanager.

ПРИЛОЖЕНИЕ Л. ЭКСПЛУАТАЦИОННЫЙ РЕГЛАМЕНТ

Л.1 Ежедневные операции

Дежурный инженер проверяет /health/ready и dashboard. Если readiness не Ready, сначала анализируются компоненты database/license/environment. При NotReady по базе проверяется CockroachDB connection string и доступность сети.

Далее выполняется diagnostics run. Если diagnosticsFreshness=fresh и enabledModules=healthyModules, система считается готовой к обычной эксплуатации. Если есть staleHealthChecks, модульные health checks выполняются повторно.

Системные логи просматриваются по компонентам Auth, Api, Modules, Diagnostics и Support. Повторяющиеся login failed записи анализируются как возможная атака или ошибка учетных данных.

При наличии новых инцидентов оператор проверяет severity и recommendations. Если создан approval request, ответственный SeniorAdmin принимает решение, а инженер не выполняет действие вне системы.

Л.2 Недельные операции

Один раз в неделю администратор проверяет production-readiness. Для production окружения статус должен быть Ready. Для development допустим Blocked, но список blockers должен быть ожидаемым и документированным.

Проверяется список enabled/disabled modules. Неиспользуемые модули должны оставаться disabled, чтобы не создавать ложные warnings и не расширять поверхность атаки. Новые модули включаются только после настройки required settings и health checks.

Проверяется срок действия лицензии. Optional gate license-expiry-window предупреждает, если срок истекает в течение 30 дней. В таком случае нужно заранее обновить license key.

Выполняется тестовая выгрузка support bundle. Проверяется, что файл открывается, содержит нужные секции и не содержит raw secrets. Это снижает риск обнаружить проблему только в момент реального инцидента.

Л.3 Действия при инциденте платформы

Если API недоступен, сначала проверяется процесс SysAssist.Api, порт, reverse proxy и health/live. Если live healthy, но ready not ready, проблема находится в зависимости: БД, лицензия или конфигурация.

Если frontend не показывает данные, проверяется VITE_API_BASE_URL, CORS, статус auth token и browser console. Ошибка 401 означает истекшую или удаленную session, 403 - недостаточные роли, 402 - лицензионную блокировку protected API.

Если модуль показывает Unhealthy, инженер открывает настройки модуля, проверяет required fields, SafeMode, UseFallbackMode и запускает Test Connection. При необходимости модуль временно отключается, чтобы не ломать общий health score.

После устранения проблемы выполняется prod-smoke.ps1. Для production обязательно использовать -RequireProductionReadiness. Результат сохраняется в журнал сопровождения вместе с correlation id и временем проверки.

ПРИЛОЖЕНИЕ М. СОСТАВ ПОСТАВКИ И КОНТРОЛЬ ВЕРСИИ

М.1 Состав поставки

Поставка SysAssist должна включать backend API, frontend static build, миграции базы данных, документацию по deployment, production-readiness checklist и smoke-скрипты. Отдельно передаются параметры окружения, но секретные значения не должны включаться в архив исходного кода или публичную документацию.

Backend поставляется как .NET приложение с зависимостями SysAssist.Domain, SysAssist.Contracts, SysAssist.Application и SysAssist.Infrastructure. Frontend поставляется как результат npm run build. База данных подготавливается миграциями EF Core, применяемыми через pipeline или операторскую задачу.

Документация поставки должна содержать README, architecture, prod-readiness, security checklist, modules SDK и deployment notes. Для дипломного проекта эти материалы находятся в локальном репозитории и отражают фактическое состояние реализации.

Контроль версии должен фиксировать номер продукта, дату сборки, commit или архивную метку, результат backend tests, frontend build и prod smoke. Без этих данных невозможно доказать, что проверенный документ соответствует фактически установленной версии.

Таблица М.1 - Состав поставки SysAssist

Компонент	Содержимое	Контроль
API	SysAssist.Api + referenced projects	dotnet build/test
Frontend	dist assets after npm run build	Vite build output
Database	EF Core migrations	Migration pipeline log
Configuration	.env.example, .env.production.example	No real secrets committed
Documentation	README, architecture, prod-readiness, security checklist	Reviewed before release

Smoke	scripts/prod-smoke.ps1	Passed with - RequireProductionReadiness
Support	support bundle endpoint	Export available and sanitized

М.2 Правила приемки версии

Версия считается принятой для production только при одновременном выполнении нескольких условий: backend tests passed, frontend build passed, health/ready Ready, diagnostics Healthy, production-readiness Ready, enabled modules Healthy, secret scan clean и support bundle sanitized.

Если хотя бы один обязательный production gate не проходит, версия может использоваться в development или demonstration, но не считается production-ready. Такой подход исключает субъективную приемку по внешнему виду интерфейса.

После приемки версии администратор фиксирует дату, ответственного, параметры окружения без секретов, список enabled modules и результат smoke. Эти сведения прикладываются к эксплуатационному журналу и могут использоваться при защите дипломного проекта.